Mercado Laboral IT en Panamá

Explicación del proyecto y del código

Análisis del mercado IT panameño con pipeline de datos, modelo estrella, Machine Learning (K-Means) y LLM. Documento de apoyo para la sustentación.

Qué hicimos y por qué — Explicación del proyecto

Documento para entender el proyecto de punta a punta: de dónde salen los datos, qué se les hizo y por qué. Escrito en dos niveles a la vez: en lenguaje sencillo (la idea) y con los términos técnicos (para que puedas sustentarlo).

1. La idea en una frase

Construimos un sistema que **recoge ofertas de empleo de tecnología en Panamá desde internet**, las **ordena y limpia**, deja que un **algoritmo descubra qué tipos de perfiles existen**, y lo muestra todo en un **dashboard interactivo de Power BI**, con ayuda de **inteligencia artificial** para resumir los datos.

En términos técnicos: es un **pipeline de datos** (ingesta → limpieza → transformación → carga) que alimenta un **modelo estrella** para BI, con una capa de **Machine Learning** (clustering) y otra de **LLM**.

2. De dónde salen los datos (la parte que más te van a preguntar)

Los datos **no los inventamos ni los descargamos de un archivo listo**. Los sacamos en vivo de **dos portales de empleo panameños**, con una técnica llamada **web scraping** (raspado web): un programa que abre la página, lee las ofertas y las guarda.

Fuente 1 — Konzerta Panamá

- Portal de empleos de Panamá.
- **Problema técnico:** la página está hecha con JavaScript (es una *SPA*, Single Page Application). Si la abres con un programa simple, ves la página vacía porque las ofertas se cargan después.
- **Solución:** usamos **Playwright**, una herramienta que abre un navegador de verdad (Chrome) por debajo, espera a que carguen las ofertas, y recién ahí las lee.
- Aportó ~295 registros.

Fuente 2 — Computrabajo Panamá

- Otro portal de empleos panameño.
- **Más fácil:** este sí manda las ofertas ya en el HTML inicial (renderizado en el servidor). Bastó con **requests + BeautifulSoup** (pedir la página y leer el HTML).
- Aportó ~114 registros.

***Por qué dos fuentes:** el enunciado pedía mínimo 2 fuentes distintas, y usar dos portales da una foto más completa del mercado. Además demuestra que el pipeline junta datos de orígenes con formatos diferentes.*

***Por qué son de Panamá:** el tema es "mercado laboral IT **en Panamá**". Ambos portales publican vacantes locales.*

3. Qué se le hizo a los datos (el pipeline, paso a paso)

Los datos crudos venían sucios y en formatos distintos. El pipeline los procesa en etapas. Piensa en una línea de producción:

Etapa 1 — Ingesta (recoger)

Los dos scrapers bajan las ofertas y las guardan tal cual en archivos crudos (`data/raw/`). Si un portal falla, el pipeline **sigue con el otro** (no se rompe todo por una fuente caída — esto se llama *degradación elegante*).

Etapa 2 — Limpieza

- Quitar ofertas sin título o repetidas.
- Normalizar el texto (quitar HTML sobrante, espacios de más, pasar a minúsculas).
- Intentar extraer el salario del texto cuando aparece.

Etapa 3 — Transformación al "esquema común"

Cada portal nombra las cosas distinto. Los unificamos a un formato único: título, empresa, ubicación, tecnologías, fecha, fuente, etc. Así una oferta de Konzerta y una de Computrabajo terminan con la misma estructura.

Etapa 4 — Extracción de tecnologías (lo que reemplaza a un "LLM" en esta parte)

Por cada oferta, un programa busca en el texto palabras de un **diccionario de ~45 tecnologías** (python, sql, azure, docker…). Es *matching* con expresiones regulares. No usa IA aquí: es búsqueda de palabras clave. Así sabemos qué tecnologías pide cada vacante.

Etapa 5 — Histórico

Cada vez que corre el pipeline, guarda una "foto" (*snapshot*) con la fecha. Por eso hay 3 fechas de captura (17, 18 y 23 de julio): son 3 corridas distintas. Esto permite ver cómo cambia la demanda en el tiempo.

Resultado del pipeline: un dataset unificado y limpio, listo para analizar.

4. El Machine Learning (el punto 1 del enunciado)

Qué hicimos

Agrupamos las ofertas por su "perfil tecnológico" usando **K-Means**, un algoritmo de **clustering** (agrupamiento).

En lenguaje sencillo

Le dimos al algoritmo todas las ofertas y le dijimos: *"agrúpalas tú solo según las tecnologías que se parecen"*. Sin decirle de antemano qué grupos hay. Él encontró **2 familias de perfiles**:

- Uno alrededor de **sql + python** (perfil más técnico/datos).
- Otro alrededor de **sql + excel** (perfil más de análisis de negocio).

En términos técnicos (para la sustentación)

- **Por qué clustering y no clasificación:** no teníamos etiquetas previas de "tipo de puesto". Con etiquetas sería aprendizaje *supervisado* (clasificación); sin ellas, es *no supervisado* (agrupamiento). K-Means es el algoritmo estándar.
- **Cómo se representa cada oferta:** como un **vector one-hot** de tecnologías (`MultiLabelBinarizer`): una casilla por cada tecnología, con 1 si la oferta la pide. Así dos ofertas parecidas quedan cerca en el espacio.

- **Cómo se eligió el número de grupos (k):** con el **método del codo** sobre la **inercia** (suma de distancias de cada punto a su centroide). Se probó k de 2 a 8; dio k = 2.
- **Técnica secundaria:** se programó una **regresión RandomForest** para predecir salario, pero no se ejecuta porque los portales panameños no publican salario en el listado (la columna quedó vacía).

Esa columna de "perfil/cluster" que descubrió el algoritmo es la que usa la **página 3 (Perfiles ML)** del dashboard.

5. El modelo estrella (el punto 2 del enunciado — la base del Power BI)

Qué es

Es la forma en que organizamos los datos para que Power BI los consulte rápido y sin repetir información. Se llama **modelo estrella** porque hay una tabla central rodeada de tablas satélite (parece una estrella).

Las tablas

- **Tabla de hechos** (`fact_ofertas`): el centro. Guarda cada oferta capturada, con números (nº de tecnologías) y **claves** que apuntan a las dimensiones.
- **Dimensiones** (las puntas de la estrella): describen. Una por cada cosa por la que queremos filtrar:
- `dim_empresa` — las empresas
- `dim_ubicacion` — las ciudades
- `dim_fuente` — el portal (konzerta / computrabajo)
- `dim_fecha` — las fechas de captura
- `dim_tecnologia` — las tecnologías, con su categoría
- `dim_cluster` — los perfiles del K-Means
- **Tabla puente** (`bridge_oferta_tecnologia`): caso especial. Como una oferta pide **varias** tecnologías y una tecnología está en **varias** ofertas (relación *muchos-a-muchos*), no se puede unir directo. La puente descompone eso en dos relaciones simples.

Por qué así y no una sola tabla gigante

- **No repite datos:** el nombre de una empresa se guarda una vez, no en cada oferta.
- **Consultas rápidas:** Power BI está optimizado para este modelo.
- **Filtros automáticos:** cada dimensión se vuelve un filtro del dashboard. Por eso al hacer clic en un portal, se filtra todo lo demás.

Todo esto se exporta a 8 archivos CSV (`data/powerbi/`) que son los que importaste en Power BI.

6. El dashboard (los puntos 3 y 4 del enunciado)

4 páginas, cada una responde preguntas distintas:

Página 1 — Panorama

El tamaño y forma del mercado: cuántas vacantes reales hay (148), cuántas empresas contratan (45), en cuántas ciudades, cuántas tecnologías se piden. Más el top de empresas y la distribución geográfica.

> ***Dato clave que debes saber explicar:*** *hay* ***409 registros*** *pero solo **148*
>
> *ofertas únicas**. La diferencia es porque el pipeline corrió 3 veces y volvió a*

*capturar las mismas ofertas. Por eso creamos la columna `Clave Oferta` (título + empresa) y contamos valores distintos, no filas. Esto se llama el **grano** de la tabla de hechos: es "oferta-captura", no "oferta".*

Página 2 — Demanda Tecnológica

Qué habilidades pide el mercado. El top: **sql, python, excel, azure, power bi**. Esto responde directo a la pregunta del proyecto. Un hallazgo: el mercado panameño demanda mucho **perfil de datos/análisis** (sql, excel, power bi), no solo desarrollo.

Página 3 — Perfiles ML

Los 2 perfiles que descubrió el K-Means, y qué tecnologías define cada uno. Conecta el dashboard con el requisito de Machine Learning.

Página 4 — Evolución e IA

Cómo cambia la demanda entre las 3 capturas, y la **Narrativa inteligente**.

Los KPIs (indicadores)

Se hicieron **casi todos con botones** (arrastrar campo → elegir agregación como "Recuento distinto" o "Promedio"), no escribiendo fórmulas. Solo una medida usó código DAX (`Concentración Top 5 Empresas`), porque rankear no existe como opción de menú.

7. El LLM / Inteligencia Artificial (el punto 5 del enunciado)

Cubierto por **dos vías**:

1. **En Power BI:** la **Narrativa inteligente** (página 4) genera sola un párrafo en lenguaje natural que resume los datos ("la tecnología X bajó 5% entre estas fechas…"). Es generación de lenguaje natural integrada.
2. **En el proyecto Python:** hay un dashboard adicional en **Streamlit** con una pestaña **"Asistente IA"** que usa un **LLM real (Ollama con el modelo llama3.2:3b, corriendo local)**. Permite hacer preguntas en lenguaje natural sobre los datos, generar un resumen ejecutivo y analizar tecnologías emergentes. El código está en `src/llm/ollama_cliente.py`.

Por eso el requisito de LLM queda cubierto aunque el visual de "Preguntas y respuestas" de Power BI no se haya usado: la Narrativa + el asistente de Ollama ya lo cumplen.

8. Resumen de cómo se cumple cada requisito del enunciado

Requisito del semestral	Cómo se cumplió
1. Técnica de ML + justificación	K-Means (clustering no supervisado) sobre tecnologías. Justificado en `JUSTIFICACION_ML.md`
2. Modelo estrella	6 dimensiones + tabla de hechos + tabla puente. Integridad verificada (0 errores)
3. Dashboards en Power BI	4 páginas interactivas con filtros y visualizaciones
4. Ampliar KPIs	6 tarjetas + medidas de participación, concentración, etc.

Requisito del semestral	Cómo se cumplió
5. LLM	Narrativa inteligente (Power BI) + asistente Ollama (Streamlit)

9. Las 5 preguntas más probables y su respuesta corta

¿De dónde sacaron los datos? De dos portales de empleo panameños (Konzerta y Computrabajo) por web scraping. Uno con Playwright (por ser JavaScript), otro con BeautifulSoup.

¿Por qué 409 registros pero 148 ofertas? El pipeline corrió 3 veces y recapturó las mismas ofertas. Contamos ofertas únicas por título + empresa, no filas.

¿Por qué K-Means? No había etiquetas de "tipo de puesto"; queríamos que los datos revelaran solos los grupos. Eso es agrupamiento no supervisado.

¿Qué es la tabla puente? Resuelve la relación muchos-a-muchos entre ofertas y tecnologías (una oferta pide varias, una tecnología está en varias).

¿Dónde está el LLM? Narrativa inteligente en Power BI + asistente con Ollama (LLM local) en el dashboard de Streamlit.

10. Preguntas reales que hizo el profesor (a otro salón)

Estas las preguntó de verdad. Estúdialas bien.

¿Qué es normalización?

En sencillo: organizar los datos para **no repetir información** y evitar errores.

Técnico, y cómo aplica al proyecto: normalizar es separar los datos en tablas de modo que cada dato se guarde **una sola vez**. En nuestro proyecto la normalización está en el **modelo estrella**: en vez de repetir el nombre de una empresa en cada oferta, lo guardamos una vez en `dim_empresa` y la tabla de hechos solo apunta a ella con un número (la clave). Lo mismo con ubicaciones, fuentes y tecnologías.

> ***Ojo — matiz importante para no contradecirte:*** *el modelo estrella normaliza las dimensiones (no repetir textos), pero* ***a propósito NO se normaliza al 100%****. Un modelo totalmente normalizado (3ª forma normal, como una base transaccional) haría las consultas de BI más lentas. La estrella busca el equilibrio: dimensiones normalizadas + una tabla de hechos central. A eso se le llama **modelo dimensional**, no una base normalizada pura. Si te preguntan, esa distinción suma.*

También hubo **normalización de texto** en la limpieza: pasar todo a minúsculas, quitar espacios y HTML sobrante, para que "Python", "python " y "PYTHON" cuenten como lo mismo.

¿Por qué la estrella? (por qué modelo estrella)

Tres razones:

1. **No repite datos** (normalización de dimensiones) → menos espacio, menos errores.
2. **Consultas rápidas** → Power BI está optimizado para este modelo; una sola tabla gigante o un modelo muy fragmentado serían más lentos.

3. **Cada dimensión es un filtro** → al hacer clic en un portal o una tecnología, todo el dashboard se filtra solo, gracias a las relaciones de la estrella.

Es el estándar de la industria para Business Intelligence.

¿Dónde se cargan los datos?

Dos niveles, según a qué se refiera:

En Power BI: `Inicio` → `Obtener datos` → `Texto/CSV`, importando los 8 archivos de la carpeta `data/powerbi/`. Se cargan por el **Editor de Power Query**, donde se revisan los tipos de dato, y con `Cerrar y aplicar` entran al modelo.

En el pipeline (Python): los datos fluyen por carpetas: `data/raw/` (crudo de los scrapers) → `data/processed/` (limpio y unificado) → `data/powerbi/` (los 8 CSV del modelo estrella). El script `run_pipeline.py` orquesta todo el flujo.

¿Qué datos usa el Power BI?

Los **8 archivos CSV** del modelo estrella (carpeta `data/powerbi/`):

- 1 tabla de hechos: `fact_ofertas` (409 registros = 148 ofertas × 3 capturas)
- 6 dimensiones: `dim_empresa`, `dim_ubicacion`, `dim_fuente`, `dim_fecha`, `dim_tecnologia`, `dim_cluster`
- 1 tabla puente: `bridge_oferta_tecnologia`

Esos CSV los generó el pipeline de Python a partir de las ofertas scrapeadas. Power BI **no se conecta a internet ni a los portales** — trabaja sobre esos CSV ya procesados.

Preguntas sobre las fórmulas del Power BI (DAX)

El profesor pregunta por las **medidas**. Esto es lo que tienes y qué hace cada una:

La mayoría de indicadores NO son fórmulas escritas, son **agregaciones**: arrastras un campo y eliges cómo se resume. Por ejemplo:

- `Ofertas Únicas` = **Recuento distinto** de `Clave Oferta` → cuenta ofertas sin repetir (da 148).
- `Empresas Contratando` = **Recuento distinto** de `id_empresa`.
- `Prom. Tecnologías por Oferta` = **Promedio** de `num_tecnologias`.
- `% con Tecnología Detectada` = **Promedio** de `tiene_tecnologia` (una columna 0/1).

Fórmulas DAX escritas (2 en total):

1. `% del Total` (participación de cada tecnología):

```
% del Total = DIVIDE(
    COUNTROWS(bridge_oferta_tecnologia),
    CALCULATE(COUNTROWS(bridge_oferta_tecnologia), ALL(dim_tecnologia))
)
```

- `COUNTROWS(bridge...)` = cuántas veces se pide esta tecnología.
- `CALCULATE(..., ALL(dim_tecnologia))` = el total de todas las tecnologías (el `ALL` quita el filtro para tener el total general).
- `DIVIDE` = la proporción. Así "sql" pasa de "77 menciones" a "12.5% de la demanda".

2. `Concentración Top 5 Empresas` (qué tan concentrado está el mercado):

```
Concentración Top 5 Empresas =
```

```
VAR Top5 = TOPN(5, VALUES(dim_empresa[nombre_empresa]), [Ofertas por Empresa], DESC)
VAR OfertasTop5 = SUMX(Top5, CALCULATE(DISTINCTCOUNT(fact_ofertas[Clave Oferta])))
VAR OfertasTotales = CALCULATE(DISTINCTCOUNT(fact_ofertas[Clave Oferta]), ALL(dim_empresa))
RETURN DIVIDE(OfertasTop5, OfertasTotales)
```

- `TOPN(5, ...)` = toma las 5 empresas que más publican.
- `SUMX` = suma las ofertas de esas 5.
- `ALL(dim_empresa)` = el total ignorando el filtro de empresa.
- `DIVIDE` = qué porcentaje del mercado concentran las 5 grandes.

Conceptos DAX que conviene nombrar:

- **Medida vs columna calculada:** una *columna* se calcula fila por fila y se guarda (ej. `Clave Oferta`); una *medida* se calcula al vuelo según los filtros del visual (ej. `% del Total`).
- `CALCULATE` = la función clave de DAX: cambia el contexto de filtro de un cálculo.
- `ALL` = quita filtros (se usa para obtener totales).
- **Contexto de filtro** = los filtros que afectan a una medida en un momento dado (segmentadores, la fila de una tabla, etc.).

> *Si te preguntan "¿por qué esta medida y no una agregación simple?": porque rankear (TOPN) o calcular un porcentaje-sobre-el-total-ignorando-el-filtro no se puede hacer arrastrando; necesita DAX. Por eso solo esas dos son fórmulas y el resto son botones.*

Explicación del código (el proyecto de Python)

Recorrido por el código que genera los datos del dashboard. Pensado para entender qué hace cada archivo sin ser el autor. En sencillo y con los términos técnicos.

Cómo está organizado el proyecto

```
mercado-it-panama/
■■■ run_pipeline.py            ← el "botón de arranque": corre todo en orden
■■■ src/
■   ■■■ config.py              ← configuración: rutas, diccionario de tecnologías
■   ■■■ ingesta/               ← los scrapers (recogen datos de internet)
■   ■   ■■■ scraper_konzerta.py
■   ■   ■■■ scraper_computrabajo.py
■   ■■■ procesamiento/         ← limpieza y armado de los datos
■   ■   ■■■ limpieza.py
■   ■   ■■■ extraccion_tecnologias.py
■   ■   ■■■ transformacion.py
■   ■   ■■■ modelo_estrella.py
■   ■■■ ml/
■   ■   ■■■ analisis.py        ← el Machine Learning (K-Means)
■   ■■■ llm/
■       ■■■ ollama_cliente.py  ← el asistente de IA (LLM)
■■■ dashboard/app.py           ← dashboard alternativo en Streamlit
■■■ data/                      ← los datos (raw → processed → powerbi)
```

La idea: cada carpeta es una etapa. Los datos entran crudos por `ingesta`, pasan por `procesamiento`, se analizan en `ml`, y salen como CSV para Power BI.

`run_pipeline.py` — el orquestador

Es el archivo que se ejecuta con `python run_pipeline.py`. No hace el trabajo pesado: **llama a los demás en orden**. Como un director de orquesta.

Su lógica, en 4 pasos:

1. **Scrapea Konzerta** (`scrapear_konzerta`)
2. **Scrapea Computrabajo** (`scrapear_computrabajo`)
3. **Procesa + ML:** une las fuentes (`construir_dataset_unificado`), corre el K-Means (`ejecutar_analisis_completo`) y guarda el histórico (`anexar_historico`)
4. **Genera el modelo estrella** (`construir_modelo_estrella`) → los 8 CSV para Power BI

***Concepto técnico:** esto es un **pipeline ETL** (Extract, Transform, Load):*

Extraer (scrapers) → Transformar (limpieza, unificación, ML) → Cargar (CSV del modelo estrella).

`src/config.py` — la configuración central

Guarda en un solo lugar las cosas que usan todos los módulos:

- **Rutas** de las carpetas (`data/raw`, `data/processed`, `data/powerbi`).

- `TECH_DICT` — el **diccionario maestro de tecnologías**: ~45 tecnologías con sus variantes de escritura (ej. `"node.js": ["node.js", "nodejs", "node"]`). Es lo que usa el detector de tecnologías.
- `TECH_CATEGORIAS` — a qué familia pertenece cada tecnología (lenguaje, framework, base de datos, cloud/devops, otro).
- **Config de Ollama** (el LLM): la URL y el modelo (`llama3.2:3b`).

> ***Por qué tener un config:*** *si mañana cambia una ruta o se agrega una tecnología, se edita* ***un solo archivo*** *en vez de buscar por todo el código. Buena práctica.*

Los scrapers (`src/ingesta/`)

Recogen las ofertas de los portales. Ambos tienen la misma estructura: abrir el sitio, leer las ofertas, guardarlas en `data/raw/`. Si fallan, **devuelven lista vacía** en vez de romper el pipeline (*degradación elegante*).

`scraper_konzerta.py`

- **Problema:** Konzerta es una *SPA* (la página se arma con JavaScript). Un lector simple vería la página vacía.
- **Solución:** usa **Playwright**, que abre un Chrome real por debajo, espera a que carguen las ofertas, y las lee con `page.evaluate()` (ejecuta JavaScript dentro del navegador para sacar los datos).
- Por cada oferta saca: título, empresa, ubicación, fecha, descripción, modalidad.
- Recorre varias páginas de resultados (paginación).

`scraper_computrabajo.py`

- **Más simple:** Computrabajo manda las ofertas ya en el HTML (*renderizado en servidor*). No hace falta navegador.
- Usa **requests** (baja la página) + **BeautifulSoup** (lee el HTML).
- Busca las tarjetas de oferta con selectores CSS (`article.box_offer`, etc.) y saca título, empresa, ubicación, salario y fecha.
- Detalle técnico: fuerza codificación UTF-8 para que los acentos ("Panamá") no se rompan.

> ***Concepto técnico:*** *web scraping. La diferencia clave entre los dos scrapers es* ***SSR vs SPA*** *(renderizado en servidor vs en el navegador). Por eso uno necesita Playwright y el otro no. Es un buen punto para la sustentación.*

El procesamiento (`src/procesamiento/`)

`limpieza.py`

Funciones para dejar el texto presentable:

- `limpiar_texto()` — quita HTML, espacios de más, pasa a minúsculas.
- `parsear_salario()` — intenta sacar números de salario de textos variados ("B/. 1,500 - 2,000", "$2500").
- `eliminar_duplicados_nulos()` — quita ofertas sin título o repetidas.

`extraccion_tecnologias.py`

El corazón de "detectar qué tecnologías pide una oferta". La función `extraer_tecnologias(texto)` recorre el `TECH_DICT` y busca cada tecnología en el texto con **expresiones regulares** (regex), respetando límites de palabra (para que "go" no se confunda dentro de "google").

> ***Importante para el 2° parcial:*** *esto* ***reemplaza a un LLM****. En vez de pedirle a una IA que extraiga las tecnologías, se hace con búsqueda de diccionario. Es más simple, gratis y explicable.*

`transformacion.py`

Toma las salidas de los dos scrapers (que tienen formatos distintos) y las **unifica a un esquema común**: mismas columnas para todas (título, empresa, ubicación, tecnologías, fecha, fuente…). Luego junta todo en un solo DataFrame y lo guarda.

También tiene `anexar_historico()`: cada corrida agrega su "foto" al histórico acumulado. Por eso hay 3 fechas de captura.

> ***Concepto técnico:*** *normalización a esquema común. Datos heterogéneos (de fuentes distintas) → un formato homogéneo. Es lo que permite analizarlos juntos.*

`modelo_estrella.py`

Convierte el dataset unificado en las **8 tablas del modelo estrella** (dimensiones + hechos + puente) y las guarda como CSV en `data/powerbi/`.

Cómo lo hace:

- Crea cada **dimensión** sacando los valores únicos (todas las empresas distintas, todas las ubicaciones…) y les da un **id** (clave *surrogate*, un número correlativo).
- Crea la **tabla de hechos** reemplazando cada texto por su id (la oferta ya no dice "Banco General" sino el id que apunta a `dim_empresa`).
- Crea la **tabla puente** oferta↔tecnología (una fila por cada par).
- Maneja los nulos con un **miembro centinela** (`id_fecha = -1` para "sin fecha").

> ***Concepto técnico:*** *dimensional modeling. Claves surrogate, tabla de hechos, dimensiones y bridge para el muchos-a-muchos. Este archivo es literalmente el que construye lo que importaste en Power BI.*

El Machine Learning (`src/ml/analisis.py`)

Hace el **clustering K-Means**. Pasos del código:

1. `preparar_features_tecnologias()` — convierte la lista de tecnologías de cada oferta en un **vector one-hot** con `MultiLabelBinarizer` de scikit-learn (una columna por tecnología, 1 o 0).
2. `_seleccionar_k_optimo()` — aplica el **método del codo**: prueba k de 2 a 8, mide la **inercia** de cada uno, y elige dónde deja de mejorar. Dio k=2.
3. `aplicar_kmeans()` — corre K-Means con ese k y etiqueta cada oferta con su cluster. Además saca las tecnologías top de cada grupo (para nombrarlos).

4. `aplicar_regresion_salario()` — la técnica secundaria (RandomForest para predecir salario). Solo corre si hay ≥20 ofertas con salario; como no las hay, se omite.

> ***Librería:*** *scikit-learn (*`KMeans`, `MultiLabelBinarizer`, `RandomForestRegressor`*).*
>
> *Es la librería estándar de ML en Python.*

El LLM (`src/llm/ollama_cliente.py`)

Conecta con **Ollama**, un programa que corre modelos de IA **localmente** (en la misma PC, sin internet ni pagar API). El modelo usado es `llama3.2:3b`.

Funciones principales:

- `consulta_natural()` — le pasas una pregunta en español y un resumen de los datos, y el LLM responde.
- `generar_resumen()` — pide al LLM un resumen ejecutivo del mercado.
- `skills_emergentes()` — compara dos capturas y pregunta al LLM qué tecnologías están creciendo.

Detalle importante: todo tiene *degradación elegante*. Si Ollama no está corriendo, las funciones devuelven un mensaje de aviso en vez de romperse (`ollama_disponible()` verifica primero).

> ***Concepto técnico:*** *LLM local vs API en la nube. Ventaja: privado y gratis.*
>
> *Se comunica por HTTP con el servicio de Ollama (*`requests.post` *al endpoint* `/api/generate`*).*

`dashboard/app.py` — el dashboard de Streamlit

Es un **segundo dashboard** (aparte del de Power BI), hecho en Python con Streamlit. Tiene dos pestañas:

- **Dashboard:** gráficas con Plotly (top tecnologías, salarios, clusters…).
- **Asistente IA:** la interfaz para hablar con el LLM (Ollama).

> *Se levanta con* `streamlit run dashboard/app.py`*. Es donde vive el LLM "real" del proyecto, y complementa al dashboard de Power BI.*

Resumen: quién hace qué

Archivo	Qué hace	Tecnología clave
`run_pipeline.py`	Orquesta todo en orden	—
`config.py`	Rutas y diccionario de tecnologías	—
`scraper_konzerta.py`	Scrapea Konzerta (SPA)	Playwright
`scraper_computrabajo.py`	Scrapea Computrabajo (SSR)	requests + BeautifulSoup
`limpieza.py`	Limpia texto y salarios	regex
`extraccion_tecnologias.py`	Detecta tecnologías (reemplaza LLM)	regex + diccionario
`transformacion.py`	Unifica al esquema común	pandas
`modelo_estrella.py`	Construye las 8 tablas para Power BI	pandas
`analisis.py`	K-Means (clustering)	scikit-learn

Archivo	Qué hace	Tecnología clave
`ollama_cliente.py`	Asistente IA	Ollama (LLM local)
`dashboard/app.py`	Dashboard alternativo + chat IA	Streamlit + Plotly